

Communications entre threads, l'exemple ProdCons (v1)

Fabrice.Kordon@lip6.fr

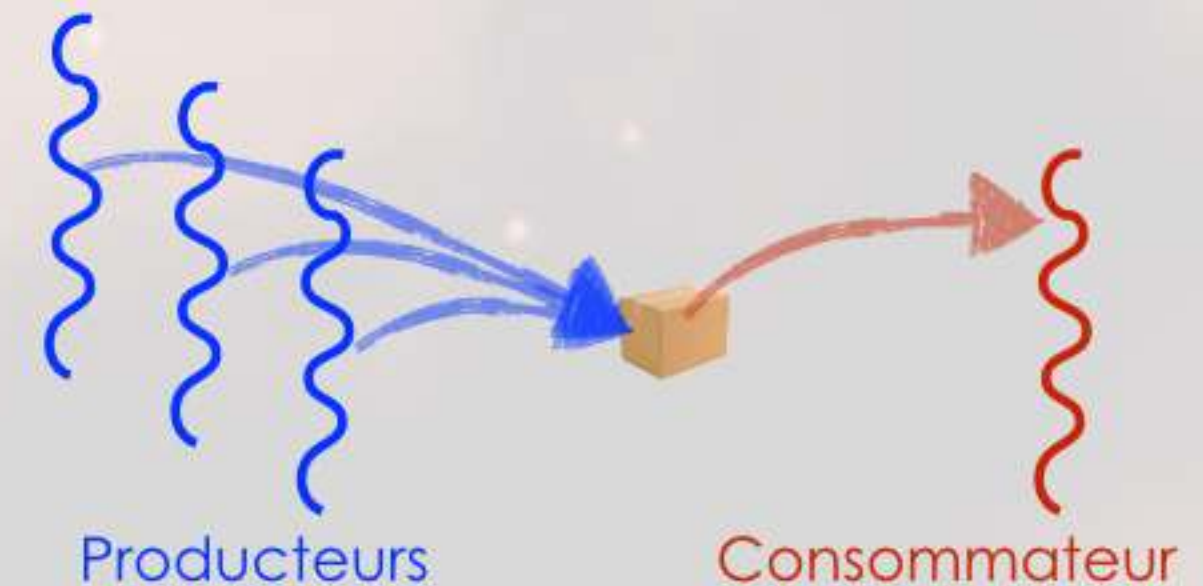


Objectif de l'exemple

2

Les acteurs

- Un consommateur (dans une thread)
- Des producteurs (chacun dans une thread)
- Une boîte d'échange
 - ▶ Les clients transmettent leur identité
 - ▶ Capacité d'un seul message



La boîte

- `estVide()`
- `estPleine()`
- `ajouteValeur(int)`
 - ▶ Peut lever une exception
- `int retireValeur()`
 - ▶ Peut lever une exception

La classe BoiteMessage

3

```
class BoiteMessage {
    boolean plein;
    int valeur;
    BoiteMessage() {
        plein = false;
    }
    synchronized public boolean estVide () {
        return ! plein;
    }
    synchronized public boolean estPleine () {
        return plein;
    }
    synchronized public void ajouteValeur (int val) throws Exception {
        if (plein) {
            throw new Exception("Boîte déjà pleine");
        }
        valeur = val;
        plein = true;
    }
    synchronized public int retireValeur () throws Exception {
        if (! plein) {
            throw new Exception("Boîte vide");
        }
        plein = false;
        return valeur;
    }
}
```


La classe Consommateur

4

```
class Consommateur implements Runnable {
    private BoiteMessage b;
    private boolean actif = true;
    Consommateur (BoiteMessage b) {
        this.b = b;
    }
    private void trace (String m) {
        System.out.println ("Serveur : " + m);
        Thread.yield(); // rendre la commutation possible
    }
    public void run() {
        try {
            this.trace ("initialisé");
            while (actif) { // boucle infinie
                if (b.estPleine()) {
                    int val = b.retireValeur();
                    this.trace ("retire " + val);
                }
            }
        }
        catch (Exception e) {
            this.trace ("probleme : " + e);
        }
        finally {this.trace ("terminé");}
    }
}
```


La classe Producteur

5

```
class Producteur implements Runnable {
    static final public int cstMaxProd = 3; // constante
    private BoiteMessage b;
    private int monNumero;
    private boolean actif = true;

    Producteur (int i, BoiteMessage b) {
        monNumero = i;
        this.b = b;
    }

    private void trace (String m) {
        String indent = String.format("%" + monNumero* 2 + "s", "");
        System.out.println (indent + "Client " + monNumero + " : " + m);
        Thread.yield(); // rendre la commutation possible
    }
}
```


La classe Producteur

5

```
class Producteur implements Runnable {
    static final public int cstMaxProd = 3; // constante
    private BoiteMessage b;
    private int monNumero;
    private boolean actif = true;

    Producteur (int i, BoiteMessage b) {
        monNumero = i;
        this.b = b;
    }

    private void trace (String m) {
        String indent = String.format("%" + monNumero* 2 + "s", "");
        System.out.println (indent + "Client " + monNumero + " : " + m);
        Thread.yield(); // rendre la commutation possible
    }

    public void run() {
        try {
            this.trace ("initialisé");
            while (actif) {
                if (b.estVide()) {
                    b.ajouteValeur(monNumero);
                    actif = false;
                }
            }
        } catch (Exception e) {
            this.trace ("probleme : " + e);
        } finally {
            this.trace ("terminé");
        }
    }
}
```


La classe principale

6

```
public class ProdCons1 {  
  
    public static void main(String []args) {  
  
        BoiteMessage boite = new BoiteMessage();  
  
        Thread cons = new Thread(new Consommateur (boite));  
        Thread prods[] = new Thread [Producteur.cstMaxProd];  
  
        cons.start(); // démarrer le serveur  
  
        for (int i = 0; i < prods.length ; i++) {  
            prods[i] = new Thread(new Producteur (i+1, boite));  
            prods[i].start(); // démarrer un client  
        }  
    }  
}
```


La classe principale

6

```
public class ProdCons1 {
```

```
$ java ProdCons1
```

```
Serveur : initialisé
```

```
    Client 3 : initialisé
```

```
    Client 1 : initialisé
```

```
    Client 2 : initialisé
```

```
Serveur : retire 3
```

```
    Client 3 : terminé
```

```
    Client 1 : terminé
```

```
    Client 2 : terminé
```

```
Serveur : retire 1
```

```
Serveur : retire 2
```

```
^C
```

```
$ java ProdCons1
```

```
Serveur : initialisé
```

```
    Client 3 : initialisé
```

```
    Client 2 : initialisé
```

```
    Client 1 : initialisé
```

```
Serveur : retire 3
```

```
    Client 3 : terminé
```

```
Serveur : retire 2
```

```
    Client 2 : terminé
```

```
    Client 1 : probleme : java.lang.Exception: Boîte déjà pleine
```

```
    Client 1 : terminé
```

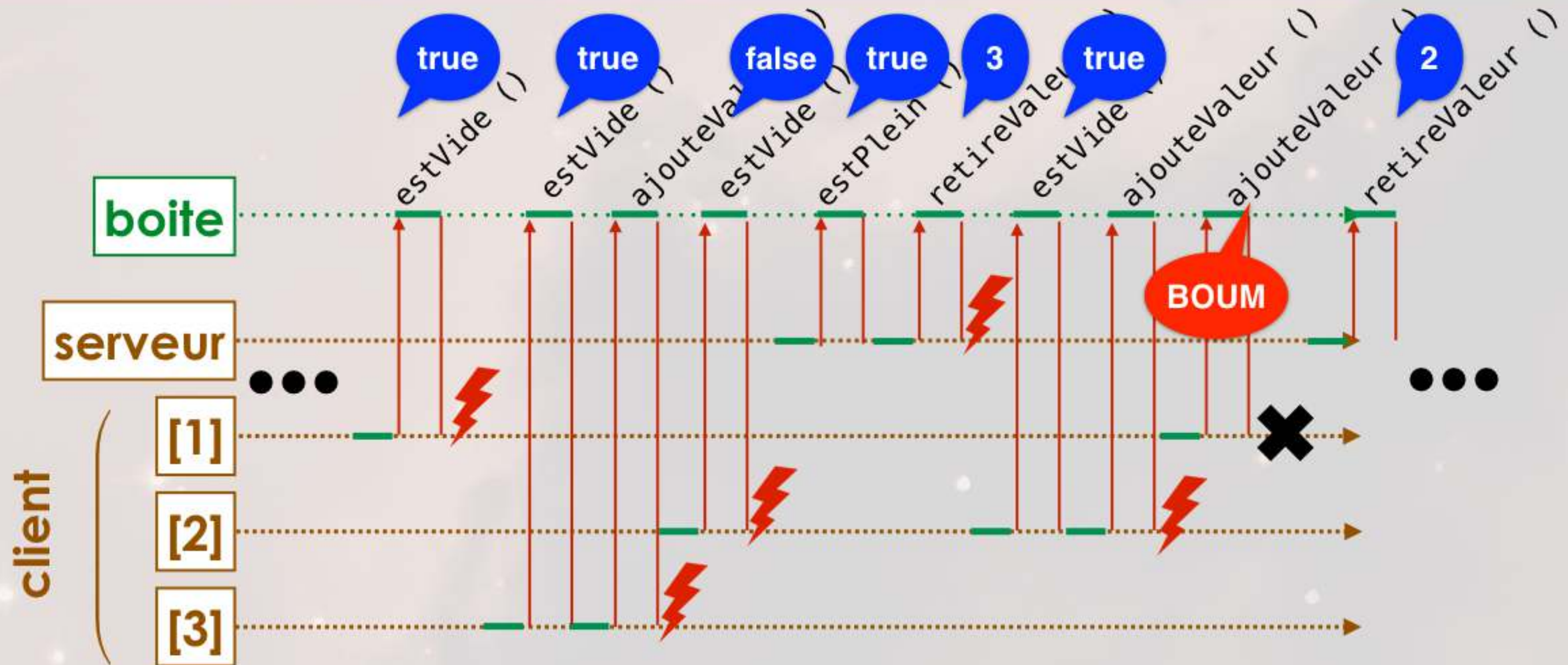
```
^C
```

Question

Que s'est-il passé?

Raisonnons sur le chronogramme

7



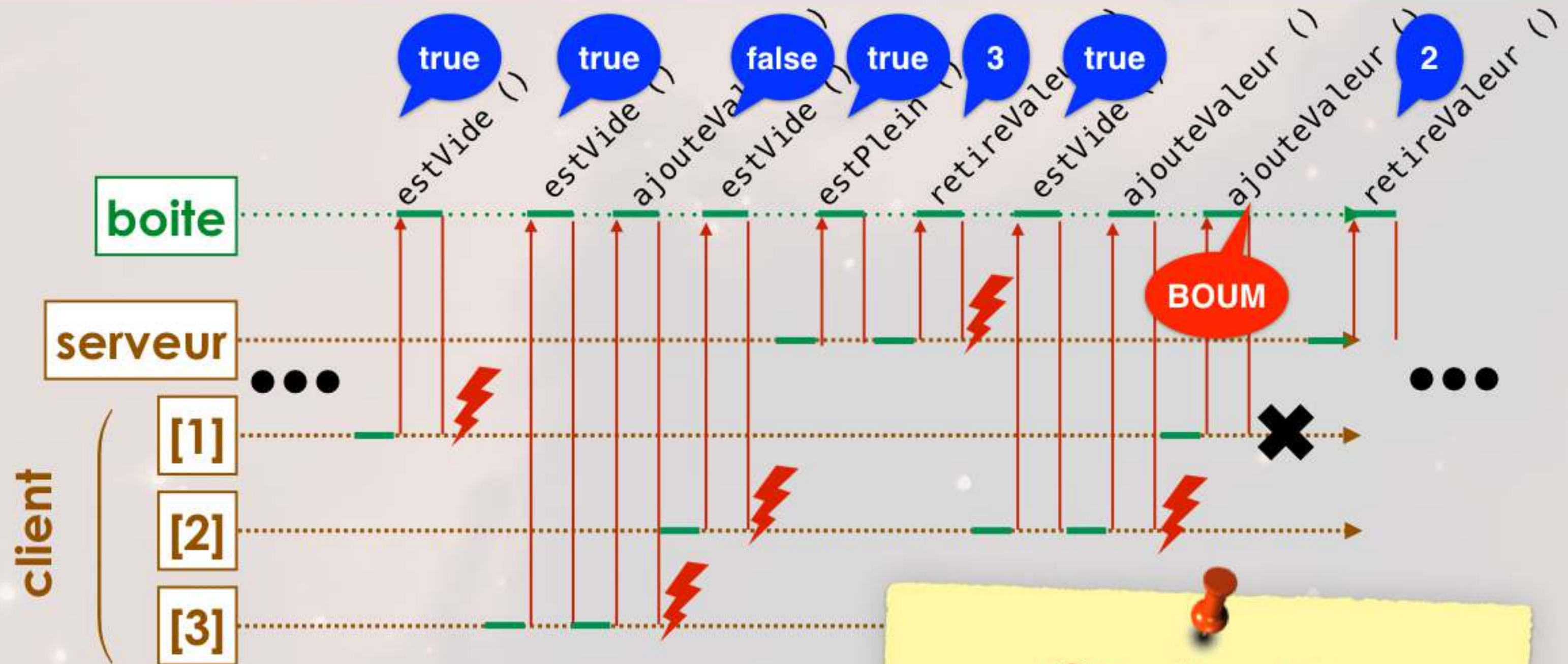
```

...
try {
  this.trace ("initialisé");
  while (actif) {
    if (b.estVide()) {
      b.ajouteValeur(monNumero);
      actif = false;
    }
  }
}
...

```


Raisonnons sur le chronogramme

7



```

...
try {
  this.t
  while
  if
}
}
}
...

```

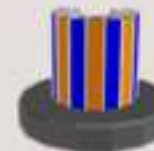
Attention
Cela arrive parfois?

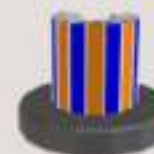
Quel est le problème?

test-and-set

En guise de conclusion

8

 L'indivisibilité, c'est vraiment délicat!

 Une règle?

 grouper comparaison et action dans la même section critique!

